

Vambe Dashboard

Documentación Técnica

26 de febrero de 2026

1. Descripción General

Dashboard de análisis de leads para Vambe AI. La aplicación permite subir un archivo CSV con datos de reuniones de venta, procesar las transcripciones con OpenAI para extraer categorías relevantes y visualizar las métricas resultantes en un panel interactivo con filtros, KPIs, gráficos y tablas.

2. Justificación del Stack Tecnológico

2.1. Next.js 16 + React 19

Next.js permite tener frontend y backend en un solo proyecto mediante API Routes. Esto simplifica el desarrollo y el despliegue, ya que no es necesario mantener un servidor aparte. Además, al ser un framework de React, permite usar componentes reutilizables y el ecosistema de React.

2.2. TypeScript

TypeScript agrega tipado estático a JavaScript, lo que permite detectar errores en tiempo de desarrollo antes de ejecutar el código. El tipado previene bugs difíciles de encontrar y mejora la experiencia de desarrollo con autocompletado.

2.3. ESLint

ESLint analiza el código en busca de patrones problemáticos (variables no usadas, imports innecesarios, etc.). Ayuda a mantener un código limpio y consistente.

2.4. Drizzle ORM

Drizzle es un ORM type-safe para TypeScript que genera queries SQL eficientes. Tiene buen rendimiento en entornos serverless (como Neon) y permite definir el schema directamente en TypeScript con soporte nativo para enums de PostgreSQL y relational queries.

2.5. Neon (PostgreSQL serverless)

Neon provee PostgreSQL serverless con conexión HTTP, ideal para proyectos desplegados en plataformas como Vercel que usan funciones serverless. No requiere mantener un pool de conexiones persistente. Se integra directamente con Drizzle mediante el driver `neon-http`.

2.6. Vercel

Vercel es la plataforma natural para desplegar Next.js. Ofrece despliegue automático desde GitHub, funciones serverless y preview deployments.

2.7. Recharts + shadcn/ui

Inicialmente se planificó usar Tremor como librería de gráficos, pero Tremor v3 no es compatible con React 19. Se migró a shadcn/ui, que integra Recharts internamente y es compatible con React 19 y Tailwind v4.

2.8. OpenAI (gpt-4o-mini)

Se eligió OpenAI porque ya contaba con créditos disponibles en la plataforma. El modelo `gpt-4o-mini` ofrece un buen balance entre costo y calidad dada la necesidad de esta app. Se envían 20 transcripciones a la vez para maximizar velocidad sin exceder rate limits.

3. Categorías extraídas por OpenAI

3.1. Proceso de definición de categorías

Las categorías y sus valores posibles fueron determinados mediante un análisis manual de las transcripciones del CSV. Se leyeron múltiples transcripciones para identificar patrones recurrentes:

- Las transcripciones mencionan industrias específicas (finanzas, retail, salud, etc.), lo que motivó el campo **industry**.
- Los clientes describen el tamaño de su operación, motivando **company_size**.
- Expresan problemas concretos (carga de trabajo, consultas repetitivas, escalabilidad), motivando **main_pain_points**.
- Mencionan cómo conocieron a Vambe (conferencia, referido, búsqueda web), motivando **discovery_source**.
- Discuten sistemas que usan o necesitan integrar (CRM, ERP, ecommerce), motivando **integrations**.
- Se identifican oportunidades de negocio (automatización, reducción de costos, expansión), motivando **opportunities**.
- El nivel de interés y urgencia varía entre leads, motivando **lead_score**.

3.2. Prompt y configuración

El prompt se almacena en `src/lib/prompt.txt` e indica al modelo a retornar exclusivamente un JSON con los campos definidos. Se usa `response_format: json_object` para forzar salida JSON válida. Además, se implementó validación post-respuesta que sanitiza valores fuera de los enums, mapeándolos a `OTHER`, ya que el LLM ocasionalmente genera valores no contemplados (ej: "PODCAST", "LEGAL").

4. Métricas y Gráficos

4.1. KPI Cards (métricas globales)

Cuatro indicadores principales visibles siempre en la parte superior:

- **Total Leads:** conteo de leads según filtros activos
- **Tasa de Conversión:** porcentaje de leads cerrados sobre el total filtrado.

- **Leads Hot:** cantidad de leads con score HOT y su porcentaje del total.
- **Leads Procesados:** progreso del procesamiento IA. Muestra barra de progreso durante el procesamiento.

4.2. Tab Pipeline

- **Leads por Mes** (AreaChart): agrupa leads por mes de reunión y muestra total y cerrados.
- **Lead Score** (Donut/PieChart): distribución de leads por score.
- **Conversión por Mes** (BarChart): tasa de conversión mensual.
- **Listado de Leads** (Tabla paginada): click abre modal con detalle completo incluyendo insights IA y transcripción. Incluye buscador.

4.3. Tab Equipo

- **Leads por Vendedor** (BarChart horizontal): conteo de leads asignados a cada vendedor.
- **Conversión por Vendedor** (BarChart horizontal): tasa de conversión de cada vendedor.
- **Score por Vendedor** (BarChart apilado): distribución HOT/WARM/COLD por vendedor.
- **Resumen por Vendedor** (Tabla): total, cerrados, tasa, HOT, WARM, COLD por vendedor.

4.4. Tab Segmentación

Incluye filtro local multi-select por industria que afecta todos los gráficos de esta tab.

- **Leads por Industria** (BarChart horizontal): conteo por industria.
- **Conversión por Industria** (BarChart horizontal): tasa de conversión por industria.
- **Tamaño de Empresa** (Donut/PieChart): distribución porcentual de tamaños. Donut para mostrar proporciones de un todo.
- **Fuente de Descubrimiento** (Donut/PieChart): cómo los leads conocieron a Vambe.

4.5. Tab Insights IA

- **Top Pain Points** (BarChart horizontal): los problemas más mencionados en transcripciones.
- **Top Oportunidades** (BarChart horizontal): oportunidades de negocio más frecuentes.
- **Integraciones Solicitadas** (BarChart horizontal): sistemas que los leads necesitan integrar.
- **Conversión por Lead Score** (BarChart agrupado): total vs cerrados por cada nivel de score.

5. Arquitectura y Decisiones Clave

5.1. Modelo de Datos

Dos tablas con relación uno-a-muchos: **sales_agent** (vendedor) y **lead**. Los campos de categorización IA usan `pgEnum`.

5.2. Filtrado client-side

Con 60 registros, el filtrado server-side agregaría complejidad innecesaria. Se trae toda la data en una llamada y se filtra en React, lo que da respuesta instantánea en los filtros. **Trade-off:** no escala a miles de registros; en ese caso se movería a queries con paginación server-side.

5.3. Procesamiento fire-and-forget

El POST `/api/process` retorna inmediatamente y procesa en background. Si el usuario recarga la página, el procesamiento continúa en el servidor. Un flag `isProcessing` previene ejecuciones duplicadas. Cada lead se actualiza individualmente para que un fallo no afecte a otros.

5.4. Polling cada 3 segundos en vez de workers

Para monitorear el progreso del procesamiento, el frontend consulta el endpoint de status cada 3 segundos. Se evaluaron alternativas como workers, pero para este contexto de prueba técnica con un solo usuario, el flag `isProcessing` en memoria cumple la misma función sin dependencias externas. Los workers serían la solución correcta en producción con múltiples usuarios.

5.5. Borrado previo al upload

Cada upload elimina datos existentes antes de insertar. Simplifica el flujo y asegura que el dashboard refleje exactamente el CSV cargado.

6. Estructura del Proyecto

```
src/
  app/
    api/
      leads/route.ts      # GET leads y status
      upload/route.ts     # POST upload CSV
      process/route.ts    # POST procesamiento IA
      page.tsx            # Dashboard principal
      layout.tsx, globals.css
    components/
      header.tsx          # Header con upload
      filters.tsx         # Filtros globales
      kpi-cards.tsx       # 4 cards de metricas
      lead-detail-modal.tsx # Modal detalle de lead
      tabs/
        pipeline-tab.tsx  # Leads por mes, conversion, tabla
        team-tab.tsx      # Metricas por vendedor
        segmentation-tab.tsx # Industria, tamaño, fuente
        insights-tab.tsx  # Pain points, oportunidades
      ui/                  # Componentes shadcn/ui
    hooks/
      use-leads.ts         # Estado, fetch, polling
      use-filters.ts       # Filtros client-side
    lib/
      db/
        schema.ts         # Schema Drizzle con enums
        index.ts          # Conexion Neon
      metrics.ts           # Funciones puras de metricas
      translations.ts      # Traducciones de enums
      openai.ts            # Integracion OpenAI + validacion
      prompt.txt           # System prompt para gpt-4o-mini
      types.ts             # Tipos TypeScript
```